

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- (a) Analyze the CSS properties responsible for layout overlapping issues.
- (b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- (c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- (d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- (e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

RESPONSIVE WEB DESIGN, JAVASCRIPT FORM VALIDATION AND CSS LAYOUT DEBUGGING

AIM

To design and implement responsive web pages using **HTML and CSS**, apply **Flexbox, Grid, media queries and box model concepts** for proper layout, and use **JavaScript with Regular Expressions** to validate email and phone number inputs with real-time error messages.

OUTPUT

Experiment 1: Responsive Layout Design

(a) Analysis of lack of responsiveness

The possible reasons for lack of responsiveness are fixed-width layouts, excessive use of absolute positioning, improper image sizing, fixed font sizes, insufficient media queries and incorrect use of margins and padding. These problems cause content to overflow or become misaligned on smaller screens.

(b) Responsive webpage using CSS Flexbox

CSS Flexbox is used to arrange webpage elements in rows or columns. The layout uses flexible widths, wrapping and proper alignment to make the header, navigation, content and footer adapt to different screen sizes.

(c) Media queries

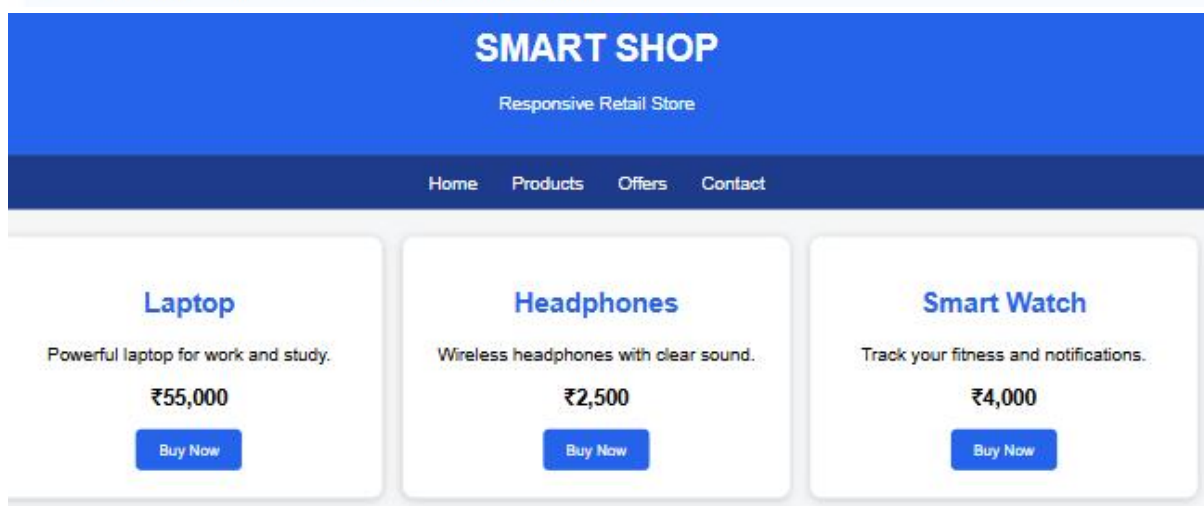
Media queries allow CSS rules to change according to the screen width. In this webpage, desktop, tablet and mobile layouts are handled using different breakpoints. On mobile screens, the navigation becomes vertical and content occupies the full width.

(d) Spacing, alignment and distribution

Flexbox properties such as gap, flex-wrap and flexible sizing are used to maintain consistent spacing and alignment. Box-sizing is also used to prevent unexpected size calculations.

(e) Evaluation and improvements

The responsive layout prevents horizontal overflow and keeps content readable on different devices. Further improvements could include responsive images, accessibility features, scalable typography and additional device breakpoints.



Experiment 2: Form Validation using JavaScript

(a) Analysis of existing validation issues

An improper validation mechanism may allow empty fields, incorrectly formatted email addresses and invalid phone numbers. Client-side validation should check the data before submitting the form.

(b) HTML input fields

HTML input elements such as `type="email"`, `type="tel"`, `required` and appropriate placeholders are used. These attributes provide basic browser-level validation and improve user input.

(c) JavaScript Regular Expressions

Regular expressions are used to check whether the entered email and phone number follow the required pattern. JavaScript processes the input and displays an error when the pattern does not match.

(d) Real-time error messages

The input event is used to validate the fields while the user is typing. Error messages are displayed immediately below the corresponding field.

(e) Evaluation and enhancements

The validation system reduces incorrect submissions and gives immediate feedback. For better security, server-side validation should also be implemented because client-side validation alone cannot be trusted.

Live Registration Form

Name

Varsha S

Email

varsha.s.puttu@gmail.com

Phone Number

7358122666

Register

Registration successful! All inputs are valid.

Experiment 3: CSS Layout Debugging

(a) Analysis of overlapping issues

Layout overlapping can occur because of improper use of `position: absolute`, fixed widths, negative margins, excessive `z-index` values and incorrect dimensions. These properties may cause elements to occupy the same area when the browser is resized.

(b) CSS Box Model

The CSS box model consists of content, padding, border and margin. The content contains the actual information, padding creates internal spacing, border surrounds the element and margin creates external spacing between elements.

(c) Positioning techniques

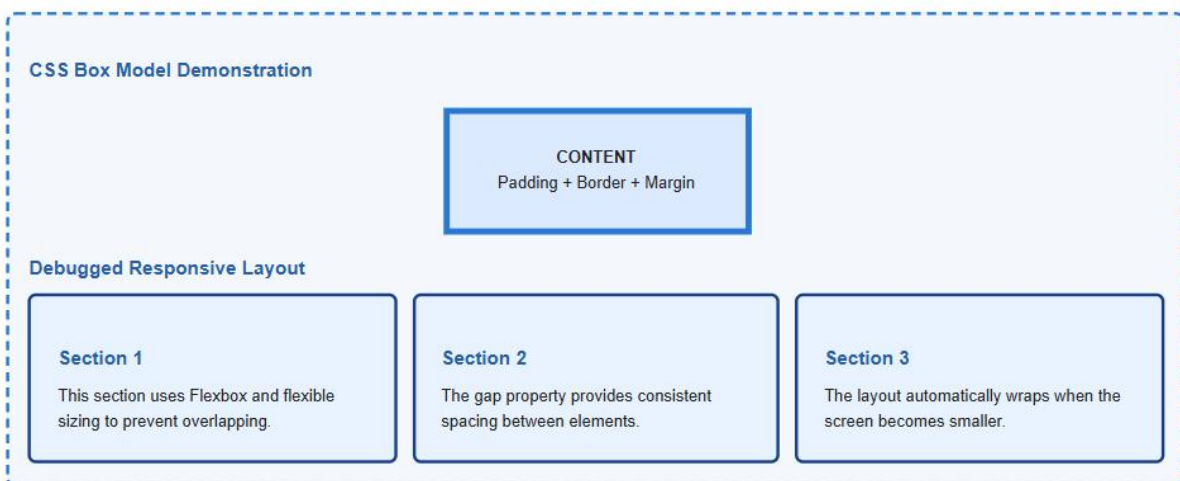
Relative positioning moves an element relative to its normal position. Absolute positioning places an element relative to its positioned parent. Flexbox and Grid are preferred for creating responsive layouts because they automatically manage alignment and distribution.

(d) Correcting the layout

The layout is corrected by using flexible widths, flex-wrap, gap and box-sizing. Instead of depending heavily on absolute positioning, Flexbox is used to arrange elements without overlap during resizing.

(e) Evaluation and best practices

The corrected layout remains organized when the browser size changes. Best practices include using responsive units, Flexbox or Grid, avoiding unnecessary fixed widths, using `box-sizing: border-box` and testing the webpage on multiple screen sizes.



RESULT

The three experiments were successfully implemented. The webpage adapted correctly to mobile, tablet and desktop screen sizes, the registration form validated email and phone numbers in real time, and CSS layout overlapping issues were corrected using Flexbox, proper spacing, box-sizing and responsive design techniques.